

# National University of Singapore

## CS2109S—Introduction to AI and Machine Learning

### MIDTERM ASSESSMENT

Semester 1, 2022/2023

**Time allowed:** 2 hours

---

### INSTRUCTIONS TO STUDENTS

1. Write down your **Student Number** on the answer sheet and shade completely the corresponding bubbles in the grid for each digit or letter. **DO NOT WRITE YOUR NAME!**
2. The assessment paper contains **FIVE (5) questions** and comprises **ELEVEN (11) pages** including this cover page.
3. Weightage of questions is given in square brackets. The maximum attainable score is 100.
4. This is a **OPEN-SHEET** assessment. You are allow one A4-sized double-sided cheat-sheet.
5. You are allowed to bring a calculator, but it cannot have any form of external communication capability, i.e. not Wifi- or 4G-enabled. Mobile phones and tablets are not allowed.
6. All questions must be answered in the space provided on the answer sheet; no extra sheets will be accepted as answers.
7. You are allowed to write with pencils, as long as it is legible.
8. **Marks may be deducted** for unrecognisable handwriting and/or for not shading the student number properly.
9. You must submit only the **ANSWER SHEET** and no other documents. The question set may be used as scratch paper.
10. An excerpt of the question may be provided in the answer sheet to aid you in answering in the correct box, where applicable. It is not the exact question. You should still refer to the original question in this question booklet.

This page is intentionally left blank.

It may be used as scratch paper.

## Question 1: Number Addition Puzzles [38 marks]

Your young cousin Ben Bitdiddle comes to you with the following logic puzzle that he found online:

$$\begin{array}{rcccc} & S & E & N & D \\ + & M & O & R & E \\ \hline M & O & N & E & Y \end{array}$$

Your goal was to find a set of digits (between 0 and 9) that would make this relationship hold. Then you realize that there are infinitely many variants for this problem. For example, one could also be asked to solve for the following:

$$\begin{array}{rcccc} & S & A & V & E \\ + & M & O & R & E \\ \hline M & O & N & E & Y \end{array}$$

In general, such addition puzzles are set up as the sum of 2 numbers, each with up to  $n$  digits, and an answer of up to  $n + 1$  (or sometimes  $n$ ) digits. The goal is to find a mapping for the  $m$  letters in the puzzle to the digits 0 to 9 so that the addition is valid. All the letters are unique digits (i.e., there is no repetition), so there can be at most 10 distinct letters in a puzzle ( $m \leq 10$ ).

Instead of simply solving the original SEND-MORE-MONEY puzzle, you decided to do better, so that your cousin won't come bother you again. You will write him program that can solve **all** such puzzles.

### Uninformed Search

- A.** Propose a state representation for this problem if we want to formulate it as a search problem and define the corresponding actions. [4 marks]
- B.** What is the invariant for your state representation in Part (A) above? In other words, what are the condition(s) that the state representation must satisfy, in order to be valid? [2 marks]
- C.** What are the initial and goal states for the problem under your proposed representation in Part (A)? [4 marks]
- D.** Which of the following statement(s) is/are true given your definition of the search problem in Parts (A) to (C)? Shade all that is/are true. [4 marks]
  - The search tree is finite.
  - There are possibly many repeated states.
  - There are possibly multiple goal states.
  - We can always find an answer (valid mapping) if we employ the right search algorithm.
  - None of the above.

**E.** Suppose we decide to apply TREE-SEARCH (See Appendix) using one of the following algorithms:

1. Depth-first search (DFS)
2. Breath-first search (BFS)
3. Depth-limited search (DLS)
4. Iterative-deepening search (IDS)

Which of the above search algorithms should we use? Explain. [4 marks]

**F.** Should we have used GRAPH-SEARCH (See Appendix) instead? Explain. [2 marks]

**G.** You implemented the TREE-SEARCH algorithm in Part (E) and realized that it often runs quite slowly. What is the expected order of growth in time for your algorithm in terms of  $n$ , for a large  $n$ , where  $n$  is the maximum number of letters in the top 2 rows. [2 marks]

**H.** Suggest how we can to improve the TREE-SEARCH algorithm you chose in Part (E) to make it run faster. [3 marks]

## Local Search

Your other cousin Alyssa P Hacker hears about you solving the problem with Search. She has also taken CS2109S and she suggests to you that you can also formulate a solution to the problem as a local search problem.

Recall that to formulate the solution as a local search problem, there are 3 steps:

1. Define a candidate solution
2. Define a transition function to generate new candidate solutions
3. Define a heuristic to evaluate the “goodness” of a candidate solution.

All 3 need to be designed in tandem because they have an impact on each other.

**I.** Based on the description of the addition puzzle above, define an initial candidate solution. [2 marks]

**J.** Define a reasonable heuristic function to evaluate the “goodness” of a candidate solution. Explain how this heuristic can also be used as a goal test to determine that we have a solution to the problem. [4 marks]

**K.** Define a reasonable transition function to generate new candidate solutions. [4 marks]

**L.** Which is the better approach: informed search or local search? Explain. [3 marks]

## Question 2: Sliding Coins [23 marks]

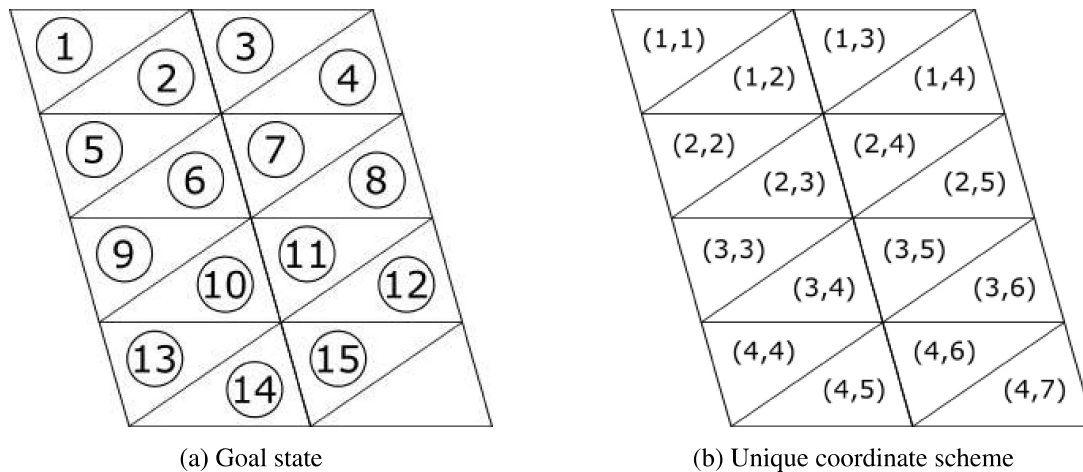
Recall the Sliding Puzzle problem covered in the lecture, where tiles begins in some arrangement with one open space. In this question, we work with a variant of the Sliding Puzzle problem where the spaces are now triangular instead of square.

We have 16 triangular spaces, and fifteen coins numbered from 1 to 15. At each time-interval, a single numbered coin can move into a open space if the two spaces are adjacent (share edges). Each time-step has a cost of 1.

In this problem, the goal is to arrange the coins in sequence from 1 to 15 from left-to-right, top-to-bottom, and finally the leaving the final space at the bottom-right open. (see Fig. 1a)

In the goal state, coin 1 is in position (1, 1), and coin 15 is in position (4, 6).

Manhattan Distance (MD) is  $|x_1 - x_2| + |y_1 - y_2|$ , thus,  $MD((1, 1), (4, 6)) = |1 - 4| + |1 - 6| = 8$ .



**Figure 1: Sliding Coin with triangular grid**

Your cousin Ben Bitdiddle immediately suggests using the same heuristics as discussed in the lectures for the original sliding puzzle problem.

**A. [Warm Up]**  $h_1$  : number of misplaced coins

Is this heuristic admissible for the current problem? Explain.

[2 marks]

**B.**  $h_2$  : total Manhattan distance of each coin to their respective goal positions

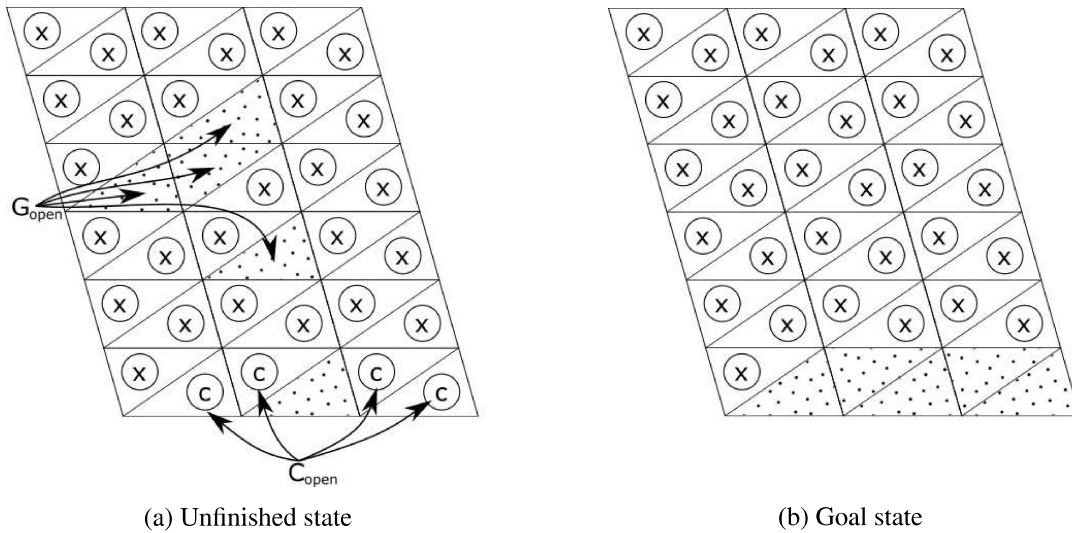
Is this heuristic admissible for the current problem? Explain.

[2 marks]

**C.** Consider a modified version of this problem where there are now  $k$  open spaces, and  $n^2 - k$  coins, where  $n^2 \gg k$ , and  $n$  is even (Figure 2). Similar to before, at each time-step, we can only move a single coin into an open space if their edges are shared. In this  $k$ -blank variant, the **coins are not numbered, so ordering is irrelevant**.

What is the maximum branching factor for this modified version of the problem? Explain.

[2 marks]



**Figure 2: Coin Sliding Puzzle where  $n^2 = 36$  and  $k = 5$**

Ben Bitdiddle has come up with the following possible heuristics to be used for this new  $k$ -open spaces problem. For each heuristic, **shade** the appropriate circle to indicate if the heuristic is admissible/consistent or not, then justify your answer in the boxes provided.

Let  $D(a, b)$  be the actual path distance between locations  $a$  and  $b$   
 and  $C_{open}$  be the set of positions of coins that are not yet in the goal positions  
 and  $G_{open}$  be the set of unoccupied/open goal positions of the coins.

- D.** Minimum of all distances of each coin to their farthest goal position. [4 marks]

$$h_A : \min_{c \in C_{open}} \{ \max_{g \in G_{open}} \{ D(c, g) \} \}$$

- E.** Sum of all distances of each coin to their closest goal position. [4 marks]

$$h_B : \sum_{c \in C_{open}} \min_{g \in G_{open}} \{ D(c, g) \}$$

- F.** Sum of all distances of each coin to their farthest goal position. [4 marks]

$$h_C : \sum_{c \in C_{open}} \max_{g \in G_{open}} \{ D(c, g) \}$$

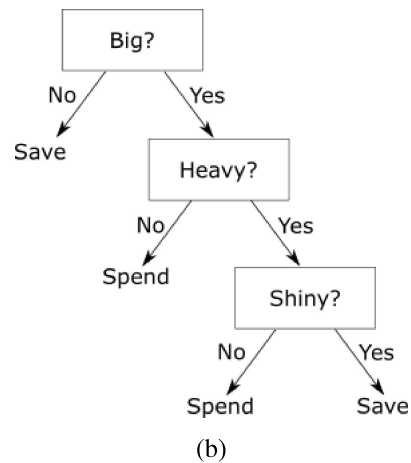
- G.** [Dominance] State and explain the dominant relationships between the 3 proposed heuristics in parts (D, E, F). [5 marks]

### Question 3: To spend or to save? [21 marks]

Your young cousin Ben Bitdiddle has to decide whether to spend or save coins in his piggy bank! To make his decision, he has drawn up a decision tree based on the following table:

| Big | Heavy | Shiny | Decision |
|-----|-------|-------|----------|
| No  | No    | No    | Save     |
| Yes | Yes   | No    | Spend    |
| Yes | No    | Yes   | Spend    |
| Yes | Yes   | Yes   | Save     |

(a)



(b)

**Figure 3: Simple decision table and associated Decision Tree**

**A. [Warm Up]** Ben is given a heavy and shiny coin. Based on the Decision Tree in Figure 3b, should Ben spend or save the coin? [2 marks]

**B.** It turns out that Ben's decision tree is not depth optimal! Help Ben optimize his decision tree by drawing a new decision tree that minimizes the depth. [2 marks]

In the end, Ben used a different set of features to decide whether to spend or save a coin. He recorded his decisions in the table below.

| Coin | Rarity | Is Vintage | Condition | In Production | Decision |
|------|--------|------------|-----------|---------------|----------|
| 1    | Rare   | No         | Rusty     | Inactive      | Spend    |
| 2    | Rare   | No         | Rusty     | Active        | Save     |
| 3    | Rare   | Yes        | Rusty     | Active        | Save     |
| 4    | Rare   | Yes        | Rusty     | Active        | Save     |
| 5    | Common | Yes        | Rusty     | Active        | Spend    |
| 6    | Common | Yes        | Rusty     | Active        | Spend    |
| 7    | Common | No         | Rusty     | Active        | Spend    |
| 8    | Common | No         | Mint      | Active        | Spend    |
| 9    | Rare   | No         | Mint      | Active        | Save     |
| 10   | Rare   | Yes        | Mint      | Inactive      | Save     |

**Table 1: Spend-or-save outcomes**

The information content for a given probability distribution  $p_i$ , for  $i = 1, \dots, n$  is given by:

$$\text{Entropy} = - \sum_{i=1}^n p_i \log_2(p_i)$$

**C.** What is the entropy of the decisions made in Table 1, rounded to 2 decimal places?

*Hint: remember the log is in base 2!*

[2 marks]

**D.** Ben Bitdiddle is a busy child, and sometimes just wants to make a quick decision. To help him, you decided to create a one-level decision tree (only one split) using information gain.

Draw the decision tree, and clearly show how your chosen feature splits the data. Demonstrate why your decision tree is correct by using the appropriate calculations of different features. You may do your calculations at 2 decimal places precision.

[6 marks]

**E.** Occasionally, Ben Bitdiddle *does* have time make rigorous decisions. In such cases, he would like to use all the information available to make his decision.

Construct the full decision tree based on the data (Table 1) using information gain to split the data. Show your working/reasoning for each node of the decision tree. Tie-breaking priority on tree construction is Rarity > Vintage > Condition > Production.

[5 marks]

**F.** [**Min-Sample Pruning**] Suppose you want a Decision Tree where each leaf corresponds to at least 3 training data points. Derive this tree by pruning the tree you obtained in Part (E). Which coin(s) do you think are likely outlier(s)/error(s)?

[4 marks]

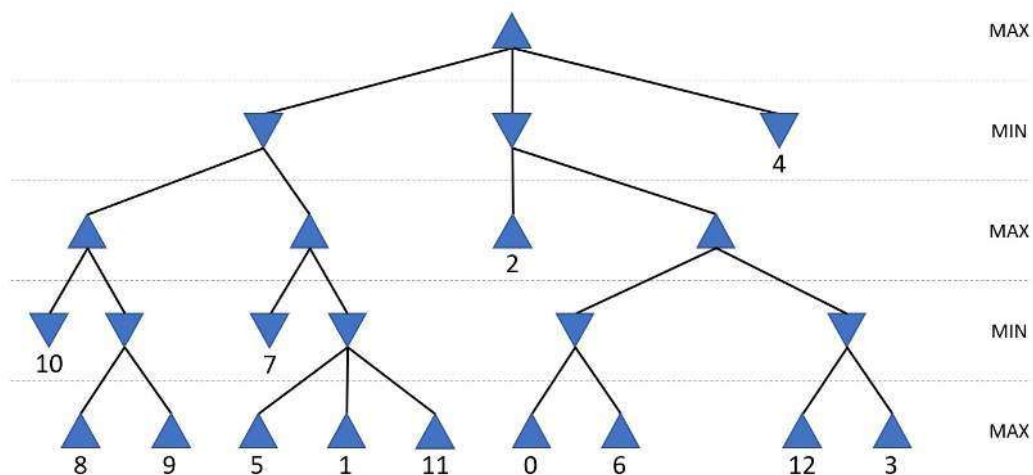
## Question 4: Solving Games [15 marks]

**A. [Warm Up]** In lecture, we discussed the following game of Nim:

Several piles of sticks are given. We represent the configuration of the piles by a **monotone** sequence of integers, such as (1,3,5). A player may remove, in one turn, any number of sticks from one pile. Thus, (1,3,5) would become (1,1,3) if the player were to remove 4 sticks from the last pile. The player who takes the last stick loses.

In this question, consider a modified game of Nim with 2 piles of sticks: one with 1 stick and one with 2 sticks. The player who takes the last stick **wins**. Draw the game tree and solve it. Which player do we expect to win? [6 marks]

**B. [Alpha-beta left-to-right]** In lecture, we discussed *Alpha-beta pruning*. Consider the following minimax tree:



Suppose we traverse this tree with DFS left to right. Cross out the links that would be pruned by *alpha-beta*. Indicate the final value of the root node. [5 marks]

**C. [Alpha-beta right-to-left]** Suppose we traverse the minimax tree in Part (B) with DFS from right to left instead. Cross out the links that would be pruned by *alpha-beta*. [4 marks]

## Question 5: What have you learnt? [3 marks]

What are the 3 most important lessons that you think you learnt in CS2109S thus far? Explain. [3 marks]

## Appendix

The following is one of the algorithms that was introduced in class that is reproduced here for your reference.

```
function TREE-SEARCH(problem, frontier) returns a solution, or failure
  frontier ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), frontier)
  loop do
    if frontier is empty then return failure
    node ← REMOVE-FRONT(frontier)
    if GOAL-TEST[problem] applied to STATE(node) succeeds return node
    frontier ← INSERTALL(EXPAND(node, problem), frontier)
```

```
function GRAPH-SEARCH(problem, frontier) returns a solution, or failure
  closed ← an empty set
  frontier ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), frontier)
  loop do
    if frontier is empty then return failure
    node ← REMOVE-FRONT(frontier)
    if GOAL-TEST(problem, STATE[node]) then return node
    if STATE[node] is not in closed then
      add STATE[node] to closed
      frontier ← INSERTALL(EXPAND(node, problem), frontier)
  end
```

```
function DECISION-TREE-LEARNING(examples, attributes, default) returns a
decision tree

  inputs: examples, set of examples
           attributes, set of attributes
           default, default value for the goal predicate

  if examples is empty then return default
  else if all examples have the same classification then return the classification
  else if attributes is empty then return MAJORITY-VALUE(examples)
  else
    best ← CHOOSE-ATTRIBUTE(attributes, examples)
    tree ← a new decision tree with root test best
    for each value  $v_i$  of best do
      examplesi ← {elements of examples with best =  $v_i$ }
      subtree ← DECISION-TREE-LEARNING(examplesi, attributes − best,
                                         MAJORITY-VALUE(examplesi))
      add a branch to tree with label  $v_i$  and subtree subtree
    end
  return tree
```

— END OF PAPER —